

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 – Experiments
		Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	Yaswanthika Shree S	Reg. No.:	192565069
Section / Batch:	B.E-CSE-Cyber Security	Date:	03.08.2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Write the algorithm and execute the code for the given experiment.
- Follow a well-structured, systematic approach to design and implement an optimal solution.
- Provide insightful analysis with accurate validation, supported by clear documentation including diagrams, code, and results.

Question Type	Focus Area	Questions
Experiments	Design and Code for Divide and Conquer Algorithms: Merge Sort, Quick Sort, Binary Search, and Matrix Multiplication	Q1

SECTION A – Experiments

Analyze Binary Search tree depth versus array-based Binary Search performance experimentally. Implement both approaches and record execution time. Interpret how tree height affects search complexity. Justify differences in performance between tree-based and array-based searches.

Code of Conduct

I Yaswanthika Shree S(192565069) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Yaswanthika Shree S

RUBRICS FOR CALCULATION

Experiments					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding ; unable to integrate concepts
Experimental Design	30%	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15%	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10%	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%				
Experimental Design	30%				
Use of Tools	20%				
Analysis of Results	15%				
Documentation	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Analyze Binary Search tree depth versus array-based Binary Search performance experimentally. Implement both approaches and record execution time. Interpret how tree height affects search complexity. Justify differences in performance between tree-based and array-based searches.



SIMATS
ENGINEERING

SIMATS Online Programming Lab
DAA PROGRAMMING

 **Yaswanthika Shree**
192565069RC

QUESTIONS

Q1
Binary Search
100 marks

0/1 answered

Q1 Binary Search 100 marks

Analyze Binary Search tree depth versus array-based Binary Search performance experimentally. Implement both approaches and record execution time. Interpret how tree height affects search complexity. Justify differences in performance between tree-based and array-based searches.

Your Code

```
1 import time
2
3 class Node:
4     def __init__(self, data):
5         self.data = data
6         self.left = None
7         self.right = None
8
9     def insert(root, key):
10        if root is None:
11            return Node(key)
12        if key < root.data:
13            root.left = insert(root.left, key)
14        elif key > root.data:
15            root.right = insert(root.right, key)
16        return root
17
18    def search_bst(root, key):
19        if root is None:
20            return False
21        if root.data == key:
22            return True
23        elif key < root.data:
24            return search_bst(root.left, key)
25        else:
26            return search_bst(root.right, key)
27
28    def binary_search(arr, key):
29        low, high = 0, len(arr) - 1
30        while low <= high:
31            mid = (low + high) // 2
32            if arr[mid] == key:
33                return True
34            elif arr[mid] < key:
```

  

Input

```
7
10 20 30 40 50 60 70
50
```

Output

```
Array Binary Search: Found
Execution Time (Array): 0.000006 sec

BST Search: Found
Execution Time (BST): 0.000009 sec
```